Verified Peephole Optimizer for Arithmetic Expressions in Lean 4

HONGXU XU, Cheriton School of Computer Science, University of Waterloo, Canada

Formal methods have been increasingly applied to verify the correctness of compiler optimizations, ensuring that transformations preserve program semantics. This report describes the implementation of lean-nano-llvm, a Lean4 formalization of a small executable LLVM-like intermediate representation (IR) with symbolic integer widths, a surface syntax embedded in Lean4, denotational semantics, structural well-formedness checks, and machine-checked refinement proofs.

Two examples illustrate the result. First, the project proves that `add x, 0` is refined by `return x` both at a fixed width and for all instantiations of a symbolic width. Second, the project models `undef` using an explicit supply of values and proves a width-generic theorem relating `add undef, undef` and `mul undef, 2`, showing that the latter is a more defined target behavior than the former. These examples capture what the project ultimately achieved well: not a broad model of all of LLVM, but a small, readable, executable framework in which subtle semantic issues such as symbolic widths, `poison`, and `undef` can already be studied with machine-checked proofs.

Additional Key Words and Phrases: Formal Methods, Compiler Optimization, LLVM, Lean4

1 Introduction

Compiler optimizations are easy to state informally and surprisingly hard to get exactly right. This is particularly true for LLVM, whose intermediate representation carries subtle semantic concepts such as `poison`, `undef`, integer overflow flags, and target-independent bitvector behavior. Production optimization passes such as InstCombine are large, heavily engineered, and known to be bug-prone [3]. Tools such as Alive2 [2] demonstrate that formal reasoning can be practical for many optimization patterns, while projects such as Vellvm [4] and LeanMLIR [1] show that proof assistants can support much deeper semantic developments.

This project explores that space from a deliberately smaller angle. The goal was not to compete with full LLVM formalizations, but to learn how to build an executable semantics and machine-checked refinement proofs in Lean4 for a carefully chosen fragment. The final artifact is lean-nano-llvm, a compact Lean4 library in which one can write small LLVM-like programs directly in Lean4, instantiate symbolic widths, execute the resulting definitions, and prove refinement theorems between definitions.

The project is best introduced through the kind of theorems it supports. One example is a width-generic algebraic identity: for every width `w`, `add i$0 %x, 0` is refined by `return %x`, where `i$0` is a symbolic width and `%x` is a symbolic variable. Another example is more semantic and interesting: for every width `w`, `add i$0 undef, undef` is refined by `mul i$0 undef, 2` in the project's explicit-`undef` model, meaning every value produced by the right-hand side is either returned exactly by the left-hand side or justified by the refinement relation's poison-aware value clause. These statements are not just comments or unit tests; they are theorems checked by Lean4 against the executable semantics.

The final system differs in several ways from the original proposal. The proposal aimed at a somewhat broader LLVM fragment, including memory operations such as `load` and `store`, and suggested building a small optimizer function and then proving it sound. Under the available time, We instead focused on constructing a coherent core: abstract syntax, embedded surface syntax, width instantiation, well-formedness, semantics, `undef` handling, and refinement proofs

Author's Contact Information: Hongxu Xu, Cheriton School of Computer Science, University of Waterloo, Waterloo, Canada, hongxu.xu@uwaterloo.ca.

for representative examples. This decision reduced scope, but it also produced a more complete and better understood semantic artifact than a partially finished optimizer would have.

The rest of this report explains what was built, how the proofs were carried out, where the design was intentionally simplified, what parts of the proposal were not completed, and what we learned from the process.

2 System Overview

The codebase is organized around four connected layers: syntax, static well-formedness, executable semantics, and refinement. Together they support the workflow "write a small LLVM-like definition, execute it, and prove that one definition is refined by another."

2.1 Abstract Syntax and Embedded Surface Syntax

The core syntax lives in the AST modules. The project defines Lean4 data types for widths, identifiers, types, expressions, instructions, terminators, blocks, and single-block function definitions. The supported instruction forms are intentionally small: integer binary operations, selected integer conversions, and `freeze`. This is enough to express the example optimizations and to surface some of the semantic issues that make LLVM optimization correctness subtle.

One design decision that paid off early was to support a concrete surface syntax inside Lean4 rather than constructing every example through raw constructors. Quasiquoters are introduced to let the theorems look very close to the LLVM syntax they are about, such as `[llvm-definition| ... ]`. The syntax layer also supports symbolic widths, written for example as `i$0`, where the `$0` is a metavariable slot. This means that a theorem can be stated once and later instantiated at any concrete width.

The syntax support is not limited to parsing. The project also includes printing and unexpansion support, so the embedded syntax is reasonably round-trippable and readable in the Lean4 environment. This matters for a project of this size: readability directly affects how easy it is to inspect the generated terms and maintain the proof scripts.

2.2 Symbolic Widths

A central technical choice was to reuse the `ConcreteOrMVar` datatype from LeanMLIR to represent widths. A width is either concrete, such as `i8`, or a metavariable slot such as `i$0`. Widths can then be instantiated by a vector of natural numbers. This design gives the project a useful middle ground. It is not fully dependent over width constraints, but it is much more expressive than hard-coding one width at a time.

This choice directly supports one of the most attractive parts of the project: width-generic theorems. Instead of proving the same identity again for `i8`, `i16`, and `i32`, we can state one theorem over a symbolic-width definition and instantiate it. This kept the project small without making it trivial.

2.3 Well-Formedness

The semantics is only intended to run on structurally sensible definitions, so we added explicit well-formedness predicates. Expressions are checked for type compatibility and local scoping. Instructions are checked against the expected operand types. Code checks enforce that instruction identifiers are defined once and then become available in scope for later instructions. Definitions check prototype shape, argument counts, duplicate argument names, instruction well-formedness, and the compatibility of the terminator with the declared return type.

This static layer is intentionally modest. It is not a full typing judgment with theorem-proved preservation properties. However, it serves two practical purposes. First, it makes the refinement

theorem state cleaner by separating semantic behavior from obviously malformed programs. Second, it prevents many proofs from devolving into low-level bookkeeping about ill-scoped or ill-typed examples.

2.4 Executable Semantics

The semantic layer interprets definitions in a state monad. The visible state currently consists only of registers, stored in a hash map from identifiers to register values. Register values are either bitvectors paired with a width or `void`.

Expressions denote integer values of a chosen width. Integer literals and booleans are embedded into bitvectors, identifiers look up register state, and `poison` propagates through the integer semantics already provided in the utility layer. Executing a definition binds its arguments, interprets the code line by line, and then interprets the return terminator.

The semantics is executable in the strong sense that a fully instantiated definition can be run from Lean4 and returns a concrete `Except` result. This turned out to be valuable well beyond testing. Because the semantic functions reduce under simplification, many small proof goals can be discharged by computation rather than by manually replaying every semantic step.

2.5 Explicit `undef` Supply

The newest and most interesting semantic feature is explicit `undef` handling. Initially, `undef` was not supported in the executable semantics. The final system introduces a separate supply state that stores a list of natural numbers together with a current index. Encountering `undef` during evaluation draws the next value from that supply and coerces it to the requested bitvector width.

This is not intended to be a perfect model of LLVM's nondeterminism. Instead, it is a pragmatic executable approximation that makes it possible to state concrete refinement theorems involving `undef` and to inspect how many `undef` choices a computation consumes. Importantly, this design forced the refinement relation to become more explicit as well, because observable behavior now depends on the chosen `undef` supply.

2.6 Refinement

The project defines refinement semantically on function definitions. In essence, `x ⊑ y` has two parts. First, if the source definition `x` is defined for every `undef` supply, then the target definition `y` must also be defined for every `undef` supply. Second, every concrete return value produced by the target definition `y` must either already be returned exactly by the source definition `x`, or be covered by a poison-aware source return, assuming both definitions are well formed, have compatible signatures, and are run on well-formed arguments.

For the current `undef` model, the definition quantifies over the `undef` supplies used to run the two sides separately. This makes the refinement relation slightly unusual, but it matches the explicit-supply execution model and was sufficient to prove the theorems in the repository.

```
def Definition.IsRefinedBy (x y : @AST.Definition φ) : Prop :=
  ∀ (args : List RegisterValue),
    AST.Definition.WellFormed x →
    AST.Definition.WellFormed y →
    Definition.SignatureCompatible x y →
    Semantics.Definition.ArgValuesWellFormed x args →
    (((∀ (u : UndefChain), ∃ (ret_x : RegisterValue),
        runNanoLlvmStateM (denoteNanoLlvmDefinition x args) default ⟨u, 0⟩ = .ok ret_x) →
      (∀ (u' : UndefChain), ∃ (ret_y : RegisterValue),
        runNanoLlvmStateM (denoteNanoLlvmDefinition y args) default ⟨u', 0⟩ = .ok ret_y)) ∧
```

```
(∀ (u : UndefChain) (ret : RegisterValue),
  runNanoLlvmStateM (denoteNanoLlvmDefinition y args) default ⟨u, 0⟩ = .ok ret →
  ∃ u',
    runNanoLlvmStateM (denoteNanoLlvmDefinition x args) default ⟨u', 0⟩ = .ok ret ∨
      ∃ ret_x,
        runNanoLlvmStateM (denoteNanoLlvmDefinition x args) default ⟨u', 0⟩ = .ok ret_x ∧
          RegisterValue.IsRefinedBy ret_x ret))
```

This definition is worth reading from top to bottom. The outer quantification ranges over concrete argument lists, together with the side conditions that both definitions are structurally well formed, have compatible signatures, and are run on well-formed source arguments. The body then splits into two semantic clauses.

The first clause is the definedness condition. If the source definition `x` can return some value for every `undef` supply, then the target definition `y` must also be able to return some value for every `undef` supply. The second clause is the value condition. It quantifies over successful target executions and asks that every concrete return value produced by `y` must either be reproduced exactly by `x`, possibly under a different `undef` supply, or be justified by a source return that refines it through the poison-aware relation on register values. This asymmetry is what allows the target to be more defined than the source without introducing any genuinely new observable behavior.

3 Constructing Machine-Checked Proofs

The core proof pattern in this project is semantic rather than syntactic. Instead of proving correctness of a global optimizer function, we state a refinement theorem directly between two definitions and unfold the semantic interpretation enough to show that whenever the target program on the right-hand side returns some value, the source program on the left-hand side can either return that same value exactly or justify it through the poison-aware register-value refinement relation. In most proofs in this repository, the exact-equality branch is enough.

3.1 A Simple Refinement: `add x, 0`

The simplest examples are algebraic identities such as:

```
theorem ret_add_x_0_is_refined_by_ret_x :
  [llvm-definition|
    define i8 @f(i8 %x) {
    entry:
      %x = add i8 %x, 0
      ret i8 %x
    }
  ] ⊑ [llvm-definition|
    define i8 @f(i8 %x) {
    entry:
      ret i8 %x
    }
  ] := by
  ...
```

The shape of the proof is straightforward but still instructive. We first introduce the refinement assumptions: arguments, return value, well-formedness of both definitions, signature compatibility, and well-formedness of argument values. Then we case-split on the input argument list to discard impossible shapes. Finally, the actual semantic goal reduces to a bitvector fact that simplification can solve.

This style scales well to a width-generic variant because width instantiation is explicit in the syntax and semantics. The theorem `ret_add_x_0_is_refined_by_ret_x_generic` shows that the same proof idea survives after replacing a fixed width by a symbolic width parameter. Here `llvm-1-definition` denotes a quasiquoter for definitions with one symbolic width parameter, later instantiated by `singletonWidths w`.

```
theorem ret_add_x_0_is_refined_by_ret_x_generic (w : Nat) :
  [llvm-1-definition|
    define i$0 @f(i$0 %x) {
    entry:
      %x = add i$0 %x, 0
      ret i$0 %x
    }
  ].instantiateWidths (singletonWidths w)
  ⊑
  [llvm-1-definition|
    define i$0 @f(i$0 %x) {
    entry:
      ret i$0 %x
    }
  ].instantiateWidths (singletonWidths w) := by
  ...
```

3.2 A More Interesting Refinement: `undef`

The more distinctive example is the theorem relating `add undef, undef` and `mul undef, 2`. In the current project this theorem is stated for every instantiated width:

```
theorem undef_add_is_refined_by_undef_mul2_generic (w : Nat) :
  [llvm-1-definition|
    define i$0 @f() {
      entry:
        %x = add i$0 undef, undef
        ret i$0 %x
    }
  ].instantiateWidths (singletonWidths w)
  ⊑
  [llvm-1-definition|
    define i$0 @f() {
      entry:
        %x = mul i$0 undef, 2
      ret i$0 %x
    }
  ].instantiateWidths (singletonWidths w) := by
  ...
```

What makes this theorem interesting is that it is not simply an algebraic identity on the same input value. The two occurrences of `undef` on the left may resolve independently, while the one occurrence on the right resolves to a single concrete value. Since the left-hand side is the source and the right-hand side is the target, the proof uses the refinement definition's existential freedom on the left-hand-side supply. Given a supply for the multiplication program, the proof constructs a matching supply for the addition program by repeating the observed choice. The arithmetic lemma then shows that doubling a bitvector by addition yields the same result as multiplication by two, so the proof can use the exact-equality branch of the refinement disjunction.

```
1   intro args hwfAdd hwfMul hsig hargs
2   constructor
3   · intro hdefined u'
4     cases args with
5     | nil ⇒
6       have hnil : False := by
7         have hrun : ... = Except.error "" := by
8           simp
9           rfl
10        obtain ⟨ret_x, hret_x⟩ := hdefined []
11        simp_all
12      exact False.elim hnil
13    | cons arg rest ⇒
14      simp [Definition.ArgValuesWellFormed] at hargs
15  · intro u hRet h
16    cases u with
17    | nil ⇒
18        have hnil : ... = Except.error "" := by
19          simp
20          rfl
21        simp_all
22    | cons x xs ⇒
23      have hmulRun : ... =
24          Except.ok (RegisterValue.bv w
25            (.value ((BitVec.ofNat w x) * (2 : BitVec w)))) := by
26        simp [simp_llvm]
27        rfl
28      refine ⟨[x, x], Or.inl, ?_⟩
29      have haddRun : ... =
30          Except.ok (RegisterValue.bv w
31            (.value ((BitVec.ofNat w x) + (BitVec.ofNat w x)))) := by
32        simp [simp_llvm]
33        rfl
34      simp_all [bitvec_double_eq_mul_two_generic]
```

The proof proceeds in four steps.

(1) At line 1, the script introduces the refinement hypotheses. The constructor immediately splits the goal into the definedness obligation at line 3 and the value-preservation obligation at line 15. Within the definedness branch, the argument list is inspected at line 4; since the function takes no arguments, the impossible nonempty case at line 13 is discharged from `Definition.ArgValuesWellFormed`.

(2) The definedness argument is resolved by contradiction in the empty argument branch at line 5. The auxiliary run fact `hrun` at line 7 shows that the source program `add undef`, `undef` fails on the empty supply. That contradicts the hypothesis `hdefined`, which claims success for every supply, so `simp_all` closes the branch.

(3) The value-preservation branch begins at lines 15 and 16. After the trivial empty-supply case, the interesting branch is the nonempty target supply `x :: xs` at line 22. The fact `hmulRun` at line 25 computes the target run as multiplication by two. The witness chosen at line 28 is the source supply `[x, x]`, forcing both occurrences of `undef` to resolve to the same concrete value; the proof then enters the left branch of the refinement disjunction, corresponding

to exact return equality. The fact `haddRun` at line 31 then computes the matching source result as `x + x`.

(4) At line 34, `simp_all` combines the execution hypothesis with the two concrete run facts and reduces the remaining goal to the arithmetic identity `BitVec.ofNat w x + BitVec.ofNat w x = BitVec.ofNat w x * 2`, which is discharged by the helper lemma below.

For reference, the helper lemma used in the final step is:

```
private theorem bitvec_double_eq_mul_two_generic {w : Nat} (b : BitVec w) :
    b + b = b * (2 : BitVec w) := by
  ...
```

Its proof is routine bitvector arithmetic and is omitted from the report.

This example captures an important lesson from the project: once `undef` is present, even very small optimization theorems stop being purely local rewrites. The proof has to say something precise about how nondeterministic choices line up between the two sides. Making that alignment explicit was one of the main conceptual advances of the final stage of the project.

The generic story is also sharp in the opposite direction. For positive widths, the converse refinement fails:

```
theorem undef_mul2_is_not_refined_by_undef_add_generic (w : Nat) (hpos : 0 < w) :
  ¬ ([llvm-1-definition|
    define i$0 @f() {
      entry:
        %x = mul i$0 undef, 2
        ret i$0 %x
    }
  ].instantiateWidths (singletonWidths w)
  ⊑
  [llvm-1-definition|
    define i$0 @f() {
      entry:
        %x = add i$0 undef, undef
        ret i$0 %x
    }
  ].instantiateWidths (singletonWidths w)) := by
  ...
```

The counterexample is simple. The addition program can return `BitVec.ofNat w 1` using the supply `[1, 0]`, because the two `undef` occurrences are chosen independently. Any execution of `mul undef, 2`, however, returns a value of the form `b * 2`, and the helper lemma `bitvec_mul_two_ne_one_generic` shows that such a result can never equal `1` when `0 < w`. So the asymmetry in the refinement relation is not an artifact of a fixed-width example; it persists uniformly across all positive bitwidths.

4 Discussion

In this section we reflect on the design decisions and trade-offs that shaped the project. We also compare the delivered artifact to the original proposal and explain why they differ.

4.1 Design Decisions

4.1.1 Focus on Definitions, Not an Optimizer Pass. A major shift was to focus on proving relationships between concrete definitions instead of implementing an optimizer function first. This made the project more semantic and less algorithmic than originally planned. In retrospect, that was the

right choice. The hard part was not writing a toy rewrite function. The hard part was deciding what its correctness should mean in the presence of widths, `poison`, and `undef`, and then making that notion executable enough to support proofs.

By proving refinement directly between definitions, we could separate the semantic core from the eventual question of how to enumerate or apply optimization patterns. A future optimizer can be built on top of this infrastructure instead of in parallel with it.

4.1.2 Executable Simplification of `undef`. Modeling `undef` was the most delicate design choice. A faithful model of full LLVM nondeterminism is not easy. The explicit supply model is a simplification, but it gave the project something valuable: a concrete and inspectable operational account of how `undef` choices are consumed. It also made it possible to formulate and prove the width-generic `undef` theorem that is now one of the strongest examples in the repository.

The downside is clear. The current model is not the full LLVM story. The refinement relation is correspondingly tuned to this supply-based semantics. This is a sound choice for the artifact as implemented, but it should not be oversold as a definitive formalization of LLVM `undef` behavior.

4.1.3 Major Differences from the Proposal. The proposal described a broader subset of LLVM, including memory operations, a small optimizer function, and proof obligations closer to "`optimize p` is refined by `p`." The delivered system is narrower. It has no memory model, no `load` or `store`, no `select`, no multi-block control flow, and no standalone optimizer pass.

This was not an accident or a loss of interest. It was a response to what the project actually demanded once the implementation started. Even a tiny LLVM-like semantics is full of choices: value representations, width handling, scoping, the role of `poison`, the shape of `undef`, and the form of the refinement theorem. Building these pieces carefully took more effort than the proposal made visible. We chose to finish a coherent core artifact rather than spread the time thinly across a larger feature list.

4.2 What Was Achieved and What Was Not

The main achievements are:

- an abstract syntax for a small LLVM-like IR with symbolic widths;
- Lean4 quasiquoters and unexpansion support for readable examples;
- explicit well-formedness checks for definitions and arguments;
- executable denotational semantics for the implemented fragment;
- an explicit-supply model of `undef`;
- a semantic refinement relation over definitions; and
- machine-checked refinement theorems, including width-generic examples.

The main missing pieces relative to the proposal are equally important to say plainly:

- no memory model;
- no `load`, `store`, or pointer reasoning;
- no optimizer function that traverses programs and rewrites them;
- no proof that such an optimizer is globally sound;
- no control flow beyond a single basic block; and
- no fully faithful treatment of all LLVM undefined-behavior corners.

We view this as a successful partial completion rather than a failed broad one. The delivered artifact is smaller than proposed, but it is real, executable, and logically integrated.

5 Lessons Learned

This project clarified a few points that were not obvious at the proposal stage. The main lessons concern where the real complexity lives, which design choices most improved proof development, and how scope control affected the quality of the final artifact.

Semantic Modeling. The first lesson is that semantic modeling dominates everything else. Before this project, we expected the interesting work to be mostly in proving algebraic rewrites. In practice, the deeper work was deciding what counts as a program, what counts as a well-formed input, what exactly execution returns, and how refinement should quantify over semantic effects such as `undef`. Once these choices were made well, many individual proofs became much easier.

Executable Semantics. The second lesson is that executable semantics is a powerful proving aid. Because the semantics reduces by computation, small examples can often be proved by simplification after the surrounding proof obligations are arranged correctly. This reinforced an appealing idea about proof assistants: one does not always have to reason *about* an interpreter abstractly if one can also reason *with* the interpreter computationally.

Surface Syntax. The third lesson is that surface syntax matters even in a proof project. Readable theorem statements changed our willingness to experiment. It was much easier to explore optimization ideas when we could write something that looked like LLVM rather than deeply nested constructors.

Scope Control. The fourth lesson is more personal. We learned that there is value in narrowing scope early and doing so explicitly. The proposal was useful because it forced us to think ambitiously, but the implemented project improved once we accepted that finishing memory, optimizer infrastructure, and proofs all at once would likely produce a weaker result. Focusing on a smaller but tighter artifact was not merely damage control; it was a real design improvement.

6 Future Work

There are several natural next steps.

Extend the IR fragment.. The most obvious direction is to add the features from the proposal that are still missing: memory, pointers, `load`, `store`, `select`, and eventually limited control flow. Doing this well would require revisiting both the state model and the well-formedness judgments.

Implement a real optimizer layer.. The current project proves refinement between hand-written pairs of definitions. A next step is to define a syntax-directed peephole optimizer and prove that each rewrite it performs is sound with respect to the existing refinement relation. The current theorem library provides a starting point for that.

***Improve the model of* `poison` *and* `undef`..** The current `undef` supply is explicit and useful, but not fully faithful to LLVM. Likewise, `freeze` is simplified. A stronger future project would study a more principled semantics for these features and compare it against the current executable approximation.

Develop proof automation.. At the moment, proofs are still mostly hand-guided. There is room for tactics or theorem schemas that discharge routine refinement obligations automatically, especially once more optimization patterns are added.

7 Conclusion

lean-nano-llvm does not yet verify a realistic fragment of all of LLVM, and it does not implement the full optimizer-oriented roadmap set out in the proposal. What it does provide is a compact, readable, executable Lean4 formalization in which small LLVM-like programs can be written directly, instantiated at arbitrary widths, executed, and related by machine-checked refinement proofs. In particular, the project now supports examples involving symbolic widths and explicit

undef behavior, which turned out to be a richer and more educational result than a purely arithmetic toy model would have been.

References

[1] Siddharth Bhat, Alex Keizer, Chris Hughes, Andrés Goens, and Tobias Grosser. 2024. Verifying peephole rewriting in SSA compiler IRs. *arXiv preprint arXiv:2407.03685* (2024).

[2] Nuno P Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. 2021. Alive2: bounded translation validation for LLVM. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. 65–79.

[3] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN conference on Programming language design and implementation*. 283–294.

[4] Jianzhou Zhao, Santosh Nagarakatte, Milo MK Martin, and Steve Zdancewic. 2012. Formalizing the LLVM intermediate representation for verified program transformations. In *Proceedings of the 39th annual ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. 427–440.